Where the lean comes from

On the runs of ex-2.2.23, the edit that removes `difference` spills onto other ops, and the spill comes from the first two slices. At those slices the model has not yet read the context, so it meets the pull with a stand-in: either lightness loaded onto e_1 in the color embedding table, or a syntax token latched to e_1. The edit at those slices then takes the lightness away from every op. Editing only the later blocks removes `difference` with almost no spill.

Spill. The largest net drop in score on another op under the edit....

Stand-in. Something token-level that satisfies the pull where no contextual concept can...

A later finding

Part of the stand-in here comes from e_1 being a coordinate axis. Under Adam the coordinate axes of the residual stream are a privileged basis, and an un-anchored model parks its syntax tokens one-hot on them, so a syntax token latched to e_1 is that habit landing on the anchored axis. The anchor does not depend on it: the rotated-basis experiment, with the basis of the optimizer rotated so that e_1 is an ordinary direction, found that the anchor holds there as well.

Spill-by-position found that the removal and the spill share the example answers, and that a syntax token latched onto e_1 shapes the spill. The $\tau \times \lambda$_a sweep found that the spill persists with no latch at all. That left two questions: what causes the spill when there is no latch, and why the anti-subspace term does nothing about it.

Removal. How much of `difference` the edit takes out...

Latch. A syntax token embedding on e_1...

This report re-analyzes the stored checkpoints of ex-2.2.23 (24 anchored and 24 control runs, at 200 and 400 epochs) and the no-emb trials of the sweep near the recipe τ (11 trials), with no new training.

Observations

- What e_1 holds on the other ops (E1): at the example answers of other ops, e_1 holds a per-example judgement of whether that one example fits `difference`. The spill does not follow it, so the spill is not the concept firing on the wrong contexts.

Spill. The largest net drop in score on another op under the edit....

- Lightness on e_1 in the embedding table (E2): every anchored run loads lightness onto e_1 in the color embedding table, darker colors further along. The lean is about three times larger on runs with no latch, and the latch decides how the run spills.

Lean. Lightness on e_1 in the color embedding table...

Latch. A syntax token embedding on e_1...

- The lean fades with depth (E3): the lean is strongest at the embedding and fades through the blocks. It is there at 200 and 400 epochs and on the no-emb trials alike, and the readout table has none of it.
- The edit by slice (E4): editing only blocks 2 to 4 removes most of `difference` with almost no spill. Editing only the embedding gives half the removal and all the spill.

Removal. How much of `difference` the edit takes out...

Scope

This is a re-analysis with no preregistration and no gate. It covers every run of ex-2.2.23: 12 anchored runs at 400 epochs and 12 at 200, each with its control at the same model seed, scored on ex-2.2.21's held-out contexts. The prose quotes the 400-epoch runs, since they have learned every op, and the 200-epoch runs are shown beside them.

The sweep trials are the no-emb trials with τ between 0.05 and 0.3, measured on the tables and states only.

The measurements follow ex-2.2.23 and spill-by-position. The task score of an op is the expected exact match of the answer on held-out contexts of that op. Removal is the drop in the `difference` score under the edit, net of the same edit on the paired control run, as a share of the way from the clean score to the target null. Spill is the largest net drop on any other op; the selectivity criterion is 0.02.

Expected exact match. The task score, as the chance that a drawn answer agrees with a true one....

Removal. How much of `difference` the edit takes out...

Target null. The score an ideal predictor would get on `difference` contexts if it had lost that op and answered from the remaining ones.

Spill. The largest net drop in score on another op under the edit....

The full edit removes the whole e_1 component at every position and slice; E4 restricts it to a set of slices. The alignment α is the cosine of the state with e_1,[1] averaged over slices 2 to 4 where a section says so.

Alignment. How far a state points along e_1...

1 Cosine similarity...

The checkpoints are the ones the recipe was tuned on, so the seed-to-seed spread here is the spread the recipe has, with nothing held out. The slice-restricted edits of E4 are on models trained with every slice pulled, so they say where the removable part of the concept sits in those models. Only a new run can say whether a pull kept off the early slices would put the concept in the later blocks.

What e_1 holds on the other ops (E1)

Spill. The largest net drop in score on another op under the edit....

Is the spill the concept itself firing on contexts that are not `difference`? At an example answer the anchor follows the posterior given that one example (example-evidence), so a `mix` context whose first example happens to fit `difference` should have some alignment there.

Alignment. How far a state points along e_1...

It does, strongly. On the 400-epoch runs the alignment at an example answer of another op is near 0.9 when that example fits `difference` on its own and near 0.1 when it does not, on every seed, latched or not. So e_1 at the example answers holds a clean per-example judgement, and other ops' contexts get it wherever an example fits.

examples that fit	share of contexts	α at the query `=`	drop, anchored	drop, control
0	83%	0.01 (0.00 to 0.02)	0.175 (0.031 to 0.354)	0.028 (0.012 to 0.052)
1	15%	0.02 (0.00 to 0.03)	0.082 (-0.042 to 0.237)	0.020 (0.001 to 0.043)
2	1%	0.03 (0.01 to 0.06)	-0.168 (-0.311 to -0.053)	-0.017 (-0.042 to 0.006)
3	0%	0.04 (-0.01 to 0.09)	-0.166 (-0.325 to -0.038)	0.004 (-0.017 to 0.039)

***The other ops' contexts by how many of their examples fit `difference` on their own**, on the 400-epoch runs: the share of contexts, α at the query `=` (slices 2 to 4), and the drop in task score under the full edit, on the anchored run and on its control. Seed mean and range.*

But that is not what spills. The contexts with no fitting example lose the most under the edit, and the few contexts with two or three fitting examples gain a little, so the correlation between the drop and the number of fitting examples is negative on every run (about -0.18). The control loses a little under the same edit at every count, with no pattern.

So the spill comes from something the edit removes at every context of the affected op, whatever its examples say. The one place the pull reaches but no context can is the embedding table (E2).

At slices 2 to 4 the alignment at the query `=` is small on the other ops' contexts, and it rises with the number of fitting examples. That fits the query position tallying the per-example judgements, though this analysis can't tell whether it is a count or a posterior.

Lightness on e_1 in the embedding table (E2)

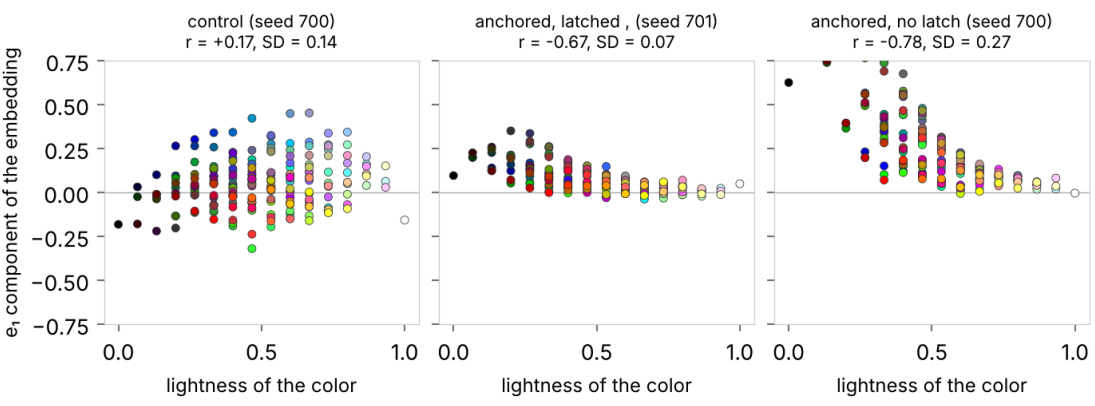
At the embedding slice the state at a position is the token embedding alone, with nothing yet read from the context. The pull asks each labeled `difference` context to have some position with α near 1 at that slice too, so it can only be met at the token level: by a latch, or by a statistical stand-in, where tokens more common in `difference` contexts sit further along e1.

Alignment. How far a state points along e1…

Latch. A syntax token embedding on e1…

Stand-in. Something token-level that satisfies the pull where no contextual concept can…

Lightness is such a stand-in. `difference` answers are the channel-wise absolute difference of the operands, so they are darker than the colors around them.

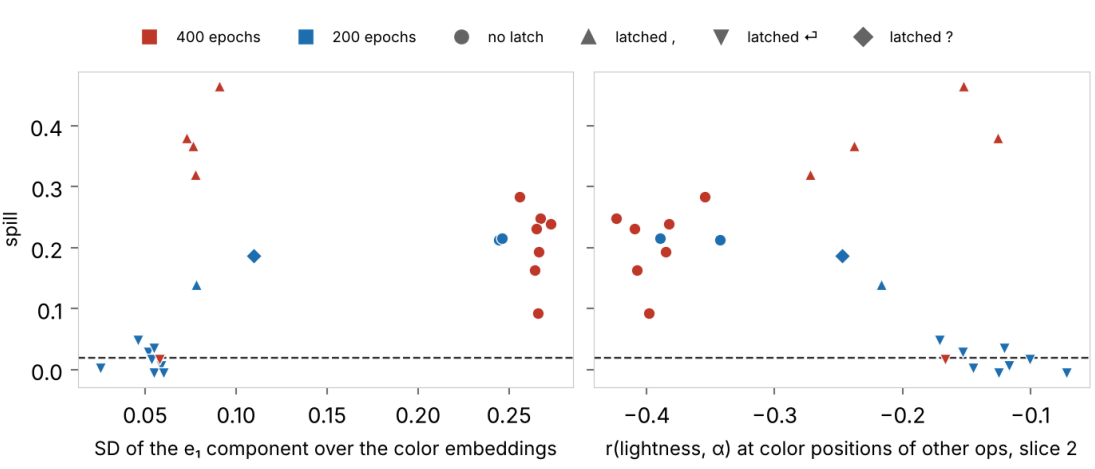


Lightness on e1 in the color embedding table, three 400-epoch runs. *Each mark is one of the 216 color embeddings, colored by the color it names; x is the lightness of the color (its mean channel), y its component along e1 after normalization. Left: a control. Middle: an anchored run latched on `,`. Right: an anchored run with no latch. Each panel's title gives the correlation and the standard deviation of the component over the table.*

Lean. Lightness on e1 in the color embedding table…

Every anchored run has this lean: the correlation between lightness and the e1 component is about -0.7 on all 24, with the same sign on every one. On the controls it averages zero but runs from -0.5 to 0.6, because lightness is a major direction of the color table and a fixed axis picks up some of it by chance, with a sign that depends on the seed. The spread of the component on the controls (0.11) is what projecting unit vectors onto a random direction in 64 dimensions gives.

The size of the lean depends on the latch. Where a syntax token sits on e1, the pull is met at that token and the color table barely moves, so the spread of the e1 component over the colors is small (0.06). Where nothing latches, the colors take the whole pull, and the spread is about three times larger (0.26).



Spill against the lean, all 24 anchored runs of ex-2.2.23. *Left: against the standard deviation of the e1 component over the color embeddings. Right: against the correlation between lightness and α at the color positions of other ops, at slice 2. Color is the training length; the marker is the latch: circles have none, up-triangles are latched on `,`, down-triangles on ⏎, and the diamond on `?`. The dashed rule is the selectivity criterion.*

Spill. The largest net drop in score on another op under the edit….

The runs latched on ⏎ barely spill, and their color table hardly leans. That token ends the context and attention stops at it, so nothing downstream reads it.

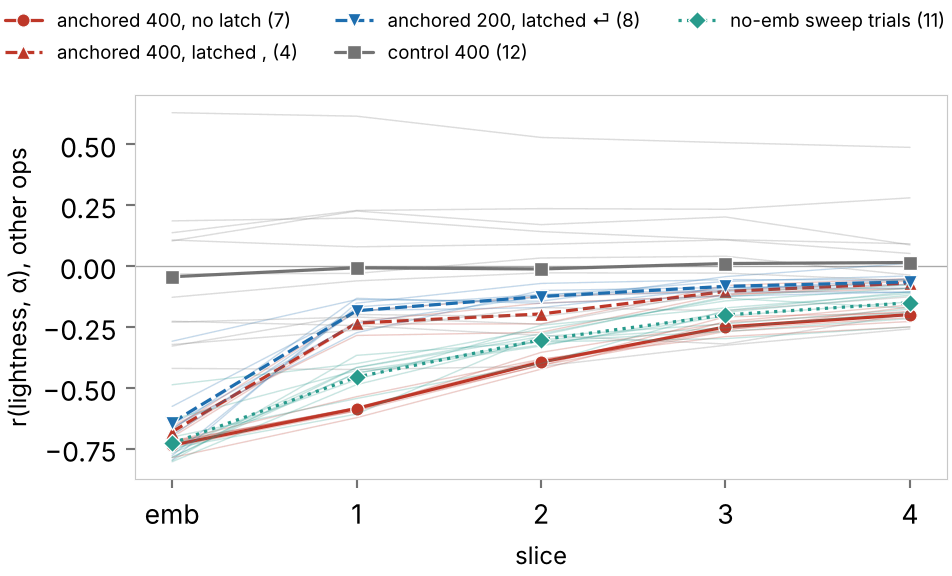
The runs latched on `,` have a small lean and the largest spill. The `,` ends every example, and on these runs nearly the whole `,` embedding lies along e1. So the projection that removes e1 removes the token itself, and every context of every op loses its example boundaries, as spill-by-position saw.

The runs with no latch have the largest lean and spill moderately, onto `darken` on all but one of them. The right panel puts the three groups on one line: the further the lightness correlation in the states has moved from zero, the more the run spills.

The lean fades with depth (E3)

If the lean is a property of the embedding table, it should be strongest at the embedding and fade as the blocks add contextual signal.

Lean. Lightness on e_1 in the color embedding ta-ble…



The lightness correlation by slice. *Each line is the seed mean of the correlation between lightness and α at the color positions of other ops, with the individual runs as faint lines behind. Circles: 400-epoch runs with no latch. Up-triangles: latched on , . Down-triangles: 200-epoch runs latched on ⏎. Squares: the controls. Diamonds: the no-emb trials of the τ × λ_a sweep, τ from 0.05 to 0.3.*

The lean is there at 200 epochs as at 400, so it comes with the pull; training longer only grows it on the runs that lose their latch. On the latched runs the lean in the states falls away at slice 1, which fits the latch taking the pull from the color table.

Latch. A syntax token embedding on e_1…

The no-emb trials lean just as far at the embedding (-0.7), although that slice is not pulled. Leaving slice 0 out of `anchor_slices` takes the anti term off it as well, so nothing holds the table in place while the pull at block 1 moves the embedding beneath it.

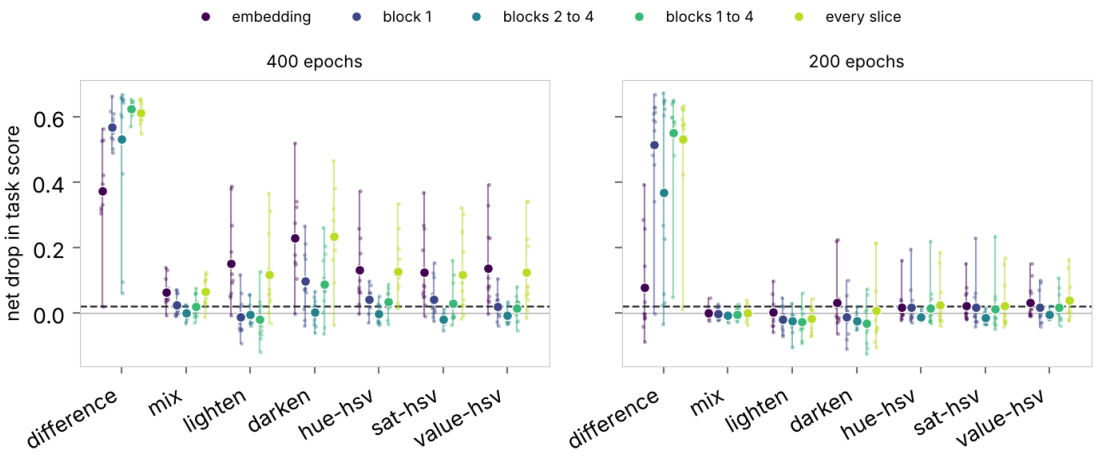
The controls spread to either side of zero at every slice, a different sign on each seed, as their embedding tables do (E2). The anchored runs all lean the same way.

The readout table has no lightness on e_1 on any run (the correlation is below 0.13 in size), so the lean is on the input side only.

The edit by slice (E4)

Is the spill caused by the edit at the early slices? Here the full edit is applied to one set of slices at a time, on every anchored run.

Spill. The largest net drop in score on another op under the edit....



The edit restricted to a set of slices, by op. *Each column is an op and each shade a slice set, from the embedding (darkest) to every slice (lightest); the large mark is the seed mean and the small marks behind it are the runs, with a thin bar spanning their range. The y axis is the drop in task score under the edit, net of the paired control. The dashed rule is the selectivity criterion.*

slices edited	removal, 400	spill, 400	removal, 200	spill, 200
every slice	0.85 (0.77 to 0.92)	0.25 (0.02 to 0.47)	0.76 (0.02 to 0.90)	0.07 (-0.01 to 0.22)
embedding	0.52 (0.03 to 0.79)	0.26 (0.00 to 0.52)	0.11 (-0.13 to 0.56)	0.06 (0.00 to 0.23)
block 1	0.79 (0.68 to 0.95)	0.12 (0.03 to 0.27)	0.74 (0.00 to 0.95)	0.05 (-0.01 to 0.23)
embedding and block 1	0.83 (0.76 to 0.92)	0.26 (0.03 to 0.51)	0.70 (-0.05 to 0.92)	0.08 (0.00 to 0.24)
blocks 1 to 4	0.87 (0.80 to 0.93)	0.12 (0.02 to 0.26)	0.79 (0.08 to 0.93)	0.05 (0.00 to 0.24)
blocks 2 to 4	0.74 (0.09 to 0.92)	0.02 (-0.01 to 0.07)	0.53 (-0.05 to 0.95)	0.01 (-0.01 to 0.03)
block 4	0.00 (-0.02 to 0.01)	0.00 (0.00 to 0.00)	-0.01 (-0.02 to 0.01)	0.00 (0.00 to 0.00)

Removal and spill under each slice-restricted edit*, seed mean and range over the 12 runs at each training length. Removal is the net drop on* `difference` *as a share of the way to the target null; spill is the largest net drop on another op.*

At 400 epochs the embedding slice alone gives half the removal and all the spill: the spill under the embedding-only edit is the same size as under the full edit, run by run. Blocks 2 to 4 alone give most of the removal, with spill within the criterion on 7 of the 12 runs.

Removal. How much of `difference` the edit takes out...

Block 4 alone does nothing. That matches the readout having no lightness on e_1, and suggests the concept is read out of the stream before the last block.

The 200-epoch runs spill little under any edit because most of them are latched on ⏎. Their removal under the late edit is lower, because the unlatched ones lean at the embedding like the 400-epoch runs.

Lean. Lightness on e_1 in the color embedding table...

But the late edit varies a lot between runs. On 3 of the 7 unlatched 400-epoch runs (seeds 700, 708, 710) the edit on blocks 2 to 4 removes much less (0.14, 0.60, 0.09) than block 1 alone does on the same runs (0.95, 0.87, 0.85), while on the other unlatched runs it removes as much as the full edit. On those runs the model has come to compute `difference` partly from the lightness that the lean put on e_1. So taking the lean away is part of how the full edit works, which makes a late-only edit on the current recipe a weak fix on its own.

Discussion

The spill has one source on these runs: the pull at the slices where no contextual concept can exist. At the embedding, and largely still at block 1, the state is the token, so the pull is met by whatever token-level feature separates `difference` contexts from the rest. The cheapest is a syntax token shared by every context (the latch); failing that, the darkness of the answers (the lean).

Spill. The largest net drop in score on another op under the edit....

Latch. A syntax token embedding on e_1...

Lean. Lightness on e_1 in the color embedding table...

The edit then removes the stand-in from every op, and the ops that depend on lightness lose accuracy. This fits the four observations and the two earlier reports, and the edit-by-slice result is the same mechanism seen from the other side.

Stand-in. Something token-level that satisfies the pull where no contextual concept can...

Two things stop the anti-subspace term from keeping the color table off e_1. The first is normalization. The anti term is the mean of $\cos^2$ over every live position and every slice, so each color embedding is one of a few thousand terms in a mean, and moving one of them along e_1 costs almost nothing.

The pull, by contrast, divides by the number of labeled contexts. Mellowmax at the recipe τ[2] also concentrates the gradient of each context on its best position, so the embedding of the darkest token in a `difference` context gets most of the pull of that context.

2 Mellowmax...

A rough count puts the pull on a color embedding an order of magnitude or more above what the anti term costs it at the hold ratio, and further above once the anti term anneals.[3] So a lean of this size is the cheapest way the model has to satisfy the slice-0 pull, and the anti term only slows it a little.

3 A rough count...

The second is that the anti weight is set relative to the anchor weight, so the sweep that varied λ_a scaled both terms together and left the ratio between them unchanged. That is why λ_a barely mattered.

The no-emb trials fit the same account: nothing holds their table in place (E3), and the edit still touches slice 0, so the lean is still removed from every op. That is also why ex-2.2.22 found that every slice restriction spilled more, with the states of other ops further along e_1 at the first two slices. The restriction took the anti term off the slices where the stand-in lives while the edit went on removing it.

If this account is right, a pull kept off the first two slices, with those slices held off e_1 by the anti term or by a hard constraint on the embedding table, would leave the model nothing to meet the pull with but a contextual feature, and the edit would then remove `difference` without the lightness.

The unlatched runs where the late edit removed little show the risk: on the current recipe the concept is partly built out of the lean, and a run with no lean has to build it another way. Whether it can is a question for a new run, since the stored checkpoints only say where the concept sits once the lean is there.

Glossary

Latch

A syntax token embedding on e_1: a token every context contains, so putting it on the anchor direction satisfies the pull at the embedding for every context at once. Found by spill-by-position.

Lean

Lightness on e_1 in the color embedding table: darker colors sit further along the anchor direction, a stand-in for `difference` at the slices where nothing contextual exists.

Stand-in

Something token-level that satisfies the pull where no contextual concept can: a latch or a lean.

Removal

How much of `difference` the edit takes out: the net drop in its score under the edit, as a share of the way from the clean score to the target null.

Spill

The largest net drop in score on another op under the edit. The selectivity criterion is 0.02.

Target null

The score an ideal predictor would get on `difference` contexts if it had lost that op and answered from the remaining ones.

---

1. Cosine similarity: how closely two vectors point the same way, ignoring length. 1 means the state lies along e_1, 0 means it is perpendicular to it. ↩

2. Mellowmax: a smooth version of the maximum over positions, here of the alignment within a context. A small τ makes it favor the best position more sharply; a large τ makes it closer to the mean. ↩

3. A rough count: at the hold the anti weight is about 0.03, spread over roughly 300 live positions per window and five slices, so one embedding at $\cos^2 = 0.3$ costs on the order of $0.03 \times 0.3 / 1500$ per window. The pull on the same embedding, where it is the best position in a labeled context, is on the order of $0.1 \times (1 - \alpha)$ / (labeled contexts per window, about 3) per window at each slice it is pulled on. The ratio is in the tens, and it depends on how many contexts the token appears in, so only the order of magnitude is meaningful. ↩